Criptoanálise de um Esquema Híbrido RSA + AES

Quebra de senha (SHA-256), fatoração RSA via GNFS e recuperação da mensagem

Grupo 2 — Gabriel Madureira e Gustavo Brendler

Algoritmos e Estruturas de Dados III (AED3) — UFPel
Prof. Ulisses Brisolara Correa
28 de outubro de 2026

Resumo

Este relatório documenta, de forma replicável, a criptoanálise completa de um desafio no tema do *Biuro Szyfrów* (a seção polonesa que quebrou a Enigma). A partir de quatro artefatos (um *hash* e três arquivos cifrados), recuperamos integralmente a cadeia: (i) quebramos o *hash* SHA-256 da senha; (ii) decifamos a chave pública RSA; (iii) fatoramos o módulo RSA de 382 bits com o *General Number Field Sieve* (CADO-NFS); (iv) reconstruímos a chave privada e decifamos a chave simétrica; e (v) decifamos a mensagem final. Registramos também as abordagens que **falharam** e sua explicação técnica, conforme exigido.

Sumário

1 Contexto e objetivo **1**

2 Arquitetura do desafio **1**

3 Etapa 1 — Quebra do *hash* da senha (SHA-256) **2**

4 Etapa 2 — Decifração da chave pública RSA **2**

5 Etapa 3 — Fatoração de N (GNFS / CADO-NFS) **2**

6 Etapa 4 — Recuperação da chave simétrica (RSA) **3**

7 Etapa 5 — Decifração da mensagem final **3**

8 Abordagens que falharam (e por quê) **4**

9 Reprodutibilidade — passo a passo **5**

10 Código-fonte **5**
10.1 `brute_combine.c` — combinador de 2 palavras (SHA-256, C+OpenMP) 5
10.2 `decrypt_pubkey.py` — decifração da chave pública 8
10.3 `factor_rsa.py` — fatoração elementar (pré-GNFS) 10
10.4 `finish.py` — reconstrução da chave privada e decifração final 11

Conclusão **13**

1 Contexto e objetivo

O enunciado situa-se no tema histórico do *Biuro Szyfrów* e da máquina Enigma. São fornecidos quatro artefatos:

- `key_for_rsa_public.hash` — *hash* SHA-256 da senha (pista: começa com `ma`);
- `key_public.en` (157 B) — chave pública RSA cifrada simetricamente com a senha;
- `key_for_message.en` (48 B) — a chave simétrica da mensagem, cifrada com RSA;
- `message.en` (1482 B) — a mensagem final, cifrada simetricamente.

O objetivo é recuperar a mensagem em claro, documentando todos os procedimentos de modo que o processo seja reprodutível a partir do zero.

2 Arquitetura do desafio

A inspeção dos arquivos (comando `file` e análise dos cabeçalhos com `xxd`) revelou a seguinte cadeia de dependências, típica de um esquema *híbrido* RSA+AES:

$$\texttt{senha} \xrightarrow{\text{SHA-256}} \textit{hash}\ \text{(alvo)}$$
$$\downarrow\ \text{(quebrar o } \textit{hash}\text{)}$$
$$\texttt{senha} \xrightarrow{\text{AES simétrico}} \text{decifra } \texttt{key_public.en} \Rightarrow \text{chave pública RSA } (N, e)$$
$$\downarrow\ \text{(fatorar } N\text{)}$$
$$N = p \cdot q \Rightarrow d = e^{-1} \bmod \varphi(N)\ \text{(chave privada)}$$
$$\downarrow$$
$$\texttt{key_for_message.en} \xrightarrow{\text{RSA } c^d \bmod N} \text{chave simétrica da mensagem}$$
$$\downarrow$$
$$\texttt{message.en} \xrightarrow{\text{AES simétrico}} \textbf{mensagem em claro}$$

Observações de cabeçalho:

- `message.en` inicia com `Salted__` (KDF do `openssl enc` com *salt*);
- `key_public.en` *não* tem `Salted__` (cifragem `-nosalt`);
- `key_for_message.en` tem exatamente 48 B = 384 bits ⇒ ciphertext RSA-384.

3 Etapa 1 — Quebra do *hash* da senha (SHA-256)

O *hash* alvo tem 64 dígitos hexadecimais (256 bits). Como o material da disciplina não ensina funções de *hash* customizadas, assumimos **SHA-256 padrão** (e validamos contra outros 6 *hashes* de 256 bits como rede de segurança). A pista é que a senha começa com `ma`.

Estratégia vencedora. Após descartar força bruta curta e dicionários simples (Seção 8), aplicamos um **combinador de duas palavras** em C+OpenMP: para cada palavra iniciada em `ma` (de um dicionário), concatenamos cada palavra de um dicionário amplo (com conectores em português opcionais), testando SHA-256 com e sem `\n` final. A senha foi encontrada:

`senha = marvinminsky`

Trata-se de **Marvin Minsky**, um dos fundadores da Inteligência Artificial (`marvin`+`minsky`); começa com `ma`, minúscula, 12 caracteres, *sem* terminador. Verificação:

```
$ echo -n "marvinminsky" | sha256sum
1b78cdb61662342d53da0544a38c87579c4ca0e9b171ad2979f165a6b3e28cce
```

que é exatamente o conteúdo de `key_for_rsa_public.hash`.

4 Etapa 2 — Decifração da chave pública RSA

Com a senha, varremos cifras/modos/KDFs do OpenSSL sobre `key_public.en` (script `decrypt_pubkey.py`). O par vencedor foi **AES-256-CTR, chave derivada por MD=SHA-256, sem *salt***:

```
$ openssl enc -d -aes-256-ctr -nosalt -md sha256 \
     -pass pass:marvinminsky -in key_public.en
-----BEGIN PUBLIC KEY-----
MEswDQYJKoZIhvcNAQEBBQADOgAwNwIwJb8QgzaQ/v8DNvt/lOiVOPgtAsRh+ZZx
6UVGq9OmaaPvjcJ4p+PmqLHCpYLLVSUbAgMBAAE=
-----END PUBLIC KEY-----
```

Extraindo os parâmetros (`openssl rsa -pubin -text -noout`):

- Módulo N (382 bits):
  `0x25bf10833690feff0336fb7f94e895d0f82d02c461f99671e94546abd3a669a3ef8dc278a7e3e6a8b1c2a582cb55251b`
- Expoente público $e = 65537$.

5 Etapa 3 — Fatoração de N (GNFS / CADO-NFS)

A segurança do RSA repousa na dificuldade de fatorar N. Com apenas 382 bits, N é *fatorável* — porém **não** por métodos elementares quando os primos são balanceados (ver Seção 8). Empregamos o *General Number Field Sieve* via **CADO-NFS**:

```
$ ./cado-nfs.py 58096943266299846783354714470120675623036278261049870981461418543170606437832625785375653806508481277012611323291 -t 4
```

Resultado (tempo real ≈ 2h39; ≈ 2h38 de CPU acumulada, 4 *threads*):

$$p = 196710398872623252246782935361034662543802850154567658817 3$$
$$q = 295342511627560754364288074964051819517730626825559272096 7$$

com $p \cdot q = N$ (ambos primos, verificado por Miller-Rabin). O *factordb* não possuía esses fatores, confirmando que a fatoração foi realizada por nós.

Montagem do *toolchain* sem privilégios de administrador. O ambiente não possuía compilador LaTeX, `pip`, `cmake`, cabeçalhos do GMP nem ferramentas de fatoração. Compilamos tudo a partir do código-fonte, em diretório do usuário: `m4` → `GMP 6.3.0` → `CADO-NFS`, obtendo o `cmake` via `pip` (ambiente virtual) e as dependências Python do CADO (`flask`, `requests`) em diretório próprio.

6 Etapa 4 — Recuperação da chave simétrica (RSA)

Com p, q reconstruímos a chave privada: $\varphi(N) = (p-1)(q-1)$ e $d = e^{-1} \bmod \varphi(N)$. Decifrando $c =$ `key_for_message.en` como inteiro *big-endian*: $m = c^d \bmod N$. O texto recuperado trazia **padding PKCS#1 v1.5** (`00 02 ...00 | payload`); após removê-lo, obtivemos a chave simétrica:

`chave da mensagem = shafigoldwas`

Referência provável a **Shafi Goldwasser** (prêmio Turing em criptografia), truncada a 12 bytes pelo tamanho do bloco RSA-384 — reforçando o tema "pioneiros da computação" também visto na senha.

7 Etapa 5 — Decifração da mensagem final

A mensagem usa `openssl enc` com *salt*. A combinação vencedora foi **AES-256-CFB, MD=SHA-256**:

```
$ openssl enc -d -aes-256-cfb -md sha256 \
     -pass pass:shafigoldwas -in message.en
```

Mensagem recuperada (1466 bytes):

```
MENSAGEM CONFIDENCIAL --GRUPO 2
============================================================

Destinatários autorizados: Gabriel Madureira, Gustavo Brendler

Classificação: ULTRA SECRETO
Protocolo: Biuro Szyfrów --Operação Enigma 2026-2

Prezados agentes,

A operação de interceptação foi bem-sucedida. Os dados coletados
confirmam que o sistema de criptografia inimigo utilizou chaves
RSA de baixa entropia combinadas com AES-256 para proteger as
comunicações. Esta combinação, embora aparentemente segura,
possui vulnerabilidades que podem ser exploradas sistematicamente.

Nossos analistas identificaram que a chave pública RSA pode ser
fatorada em tempo viável utilizando o algoritmo GNFS (General
Number Field Sieve). A senha utilizada para proteger esta chave
foi derivada de um espaço de busca limitado, permitindo ataques
de força bruta contra o hash armazenado.

Recomendações operacionais:
  1. Utilize ferramentas especializadas para recuperação da senha.
  2. Aplique algoritmos de fatoração para a chave RSA.
  3. Documente todos os procedimentos para replicabilidade.

Este documento foi gerado exclusivamente para o grupo 2
(Gabriel Madureira, Gustavo Brendler). Qualquer compartilhamento não autorizado será
detectado através da análise forense dos metadados embutidos.

Assinado digitalmente,
Biuro Szyfrów --Divisão de Criptoanálise
Data: 11/10/2026
============================================================
```

A mensagem endereça explicitamente o **Grupo 2** (Gabriel Madureira e Gustavo Brendler), confirmando o sucesso da criptoanálise.

8 Abordagens que falharam (e por quê)

Documentamos aqui as principais tentativas malsucedidas e a razão técnica de cada fracasso. (O registro completo, com mais de 30 variações, está em `TENTATIVAS.md`.)

1. **Força bruta de senha "longa" (15+ caracteres).** Guiados por um artigo de segurança indicado em aula (que recomenda *passphrases* de 15+ caracteres), chegamos a supor uma frase longa. Um espaço de $26^{13} \approx 2.5 \times 10^{18}$ (minúsculas, 13 caracteres após `ma`) a ∼27 milhões de *hashes*/s levaria ≈2900 anos. *Conclusão: inviável* — e, de fato, a senha era um **nome de**

12 caracteres, não uma *passphrase*. Lição: priorizar o *tema* (nomes de pioneiros) em vez do tamanho.

2. **Ataque "textbook RSA" à chave da mensagem (contornar a senha).** Tentamos recuperar a chave simétrica diretamente de `key_for_message.en` assumindo expoente pequeno ($m = \sqrt[e]{c}$, $e \in \{3, 5, 7, \dots\}$, *big/little-endian*), sem precisar fatorar N. Falhou: o expoente real é $e = 65537$ e $c \approx 2^{381}$ sofre redução modular por N, logo *não há raiz inteira exata*. Sem N (portanto sem a senha) não há atalho.

3. **Dicionários e *rockyou* + regras (*best64*, *leet*, anos).** Testamos ∼15 funções de *hash* sobre *rockyou* (filtrado por prefixo `ma`) e dicionários PT/EN com *mangling*. Falhou: a senha (`marvinminsky`) é a *concatenação de dois nomes*, não uma palavra isolada nem uma variação trivial — só um **combinador de 2 palavras** a alcançaria.

4. **Fatoração elementar de N (Fermat, Pollard $p-1$, Brent-ρ).** Em Python puro, resolvem apenas casos fracos (primos próximos ou com fator pequeno). Para N de 382 bits com primos *balanceados* (≈191 bits cada), o custo é proibitivo (ρ seria $\sim 2^{95}$ operações). Por isso foi necessário o **GNFS (CADO-NFS)**.

Resumo das famílias de ataque testadas. A Tabela 1 consolida, por família, as mais de 30 variações registradas em `TENTATIVAS.md` (mantido como apêndice bruto). Nenhuma das abordagens de força bruta, dicionário/*rockyou* ou *passphrase* teve êxito; somente o **combinador de duas palavras** recuperou a senha.

Família de ataque	**Espaço / ferramentas**	**Resultado**
Força bruta de prefixo `ma` ([a-z], [a-z0-9], caixa mista, símbolos; até 8–10 chars)	bilhões de *hashes*; 7 funções de 256 bits (C+OpenMP)	falhou
Dicionários + *rockyou* (`ma*`) com regras *best64*/*leet*/anos, 3 *encodings*	∼15 funções de *hash*	falhou
Passphrases/frases temáticas (Enigma/cripto; PL/DE/EN/PT; 1–3 palavras)	milhões de combinações	falhou
Nomes do grupo e dos criptoanalistas do Biuro Szyfrów	milhares de combinações	falhou
Construções alternativas (*double-hash*; entrada hex/base64/revertida; UTF-16/32)	dezenas × 7–12 *hashes*	falhou
"Textbook RSA" na chave da mensagem (expoente pequeno; contornar a senha)	$e \in \{3, \dots, 65537\}$, big/little-endian	falhou
Fatoração elementar de N (Fermat, Pollard p–1, Brent-ρ; Python)	válida só p/ primos fracos	falhou
Combinador de 2 palavras (palavra-`ma` × dicionário amplo; C+OpenMP)	SHA-256	**sucesso**

Tabela 1: Resumo das abordagens testadas na quebra da senha e na fatoração de N.

9 Reprodutibilidade — passo a passo

1. **Senha:** compilar e rodar o combinador
   `gcc -O3 -fopenmp -o brute_combine brute_combine.c`
   `./brute_combine mawords.txt allwords.txt 10` ⇒ `marvinminsky`.
2. **Chave pública:** `openssl enc -d -aes-256-ctr -nosalt -md sha256 -pass pass:marvinminsky -in key_public.en`.
3. **Extrair N, e:** `openssl rsa -pubin -in pub.pem -text -noout`.
4. **Fatorar N:** `./cado-nfs.py <N> -t 4` ⇒ p, q.

5. **Mensagem:** `python3 finish.py <p> <q>` (reconstrói a privada, decifra a chave simétrica e a mensagem).

10 Código-fonte

10.1 `brute_combine.c` — combinador de 2 palavras (SHA-256, C+OpenMP)

```c
/* Combinador de 2 palavras (passphrase) em C + OpenMP, SHA-256.
 * candidato = maword [+ conector] + word, minusculo, sem espaco.
 * Conectores: "", "de", "da", "do", "e". Terminadores: "" e "\n".
 * Uso: ./brute_combine <arq_mawords> <arq_words> [minlen]
 * Compilar: gcc -O3 -fopenmp -o brute_combine brute_combine.c
 */
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>

/* ---- SHA-256 (bloco unico, msg < 56 bytes) ---- */
static const uint32_t K[64] = {
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
    0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
    0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc,
    0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
    0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13,
    0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
    0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5,
    0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208,
    0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
};

#define ROR(x, n) (((x) >> (n)) | ((x) << (32 - (n))))

static void sha256_blk(const uint8_t *msg, size_t len, uint8_t out[32]) {
    uint8_t b[64];
    for (int i = 0; i < 64; i++)
        b[i] = 0;
    for (size_t i = 0; i < len; i++)
        b[i] = msg[i];
    b[len] = 0x80;

    uint64_t bits = (uint64_t)len * 8;
    for (int i = 0; i < 8; i++)
        b[63 - i] = (bits >> (i * 8)) & 0xff;

    uint32_t s[8] = {
        0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
        0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
    };

    uint32_t w[64];
    for (int i = 0; i < 16; i++)
        w[i] = (b[i*4] << 24) | (b[i*4+1] << 16) | (b[i*4+2] << 8) | b[i*4+3];
    for (int i = 16; i < 64; i++) {
        uint32_t s0 = ROR(w[i-15], 7) ^ ROR(w[i-15], 18) ^ (w[i-15] >> 3);
        uint32_t s1 = ROR(w[i-2], 17) ^ ROR(w[i-2], 19) ^ (w[i-2] >> 10);
```

```
        w[i] = w[i-16] + s0 + w[i-7] + s1;
    }

    uint32_t a = s[0], bb = s[1], c = s[2], d = s[3];
    uint32_t e = s[4], f = s[5], g = s[6], h = s[7];
    uint32_t t1, t2;
    for (int i = 0; i < 64; i++) {
        uint32_t S1 = ROR(e, 6) ^ ROR(e, 11) ^ ROR(e, 25);
        uint32_t ch = (e & f) ^ (~e & g);
        t1 = h + S1 + ch + K[i] + w[i];
        uint32_t S0 = ROR(a, 2) ^ ROR(a, 13) ^ ROR(a, 22);
        uint32_t mj = (a & bb) ^ (a & c) ^ (bb & c);
        t2 = S0 + mj;
        h = g; g = f; f = e; e = d + t1;
        d = c; c = bb; bb = a; a = t1 + t2;
    }

    s[0] += a; s[1] += bb; s[2] += c; s[3] += d;
    s[4] += e; s[5] += f; s[6] += g; s[7] += h;
    for (int i = 0; i < 8; i++) {
        out[i*4] = s[i] >> 24;
        out[i*4+1] = s[i] >> 16;
        out[i*4+2] = s[i] >> 8;
        out[i*4+3] = s[i];
    }
}

static const char *THEX =
    "1b78cdb61662342d53da0544a38c87579c4ca0e9b171ad2979f165a6b3e28cce";
static uint8_t TARGET[32];

static void hex2bin(const char *h, uint8_t *o) {
    for (int i = 0; i < 32; i++) {
        unsigned v;
        sscanf(h + i*2, "%2x", &v);
        o[i] = v;
    }
}

static int eq(const uint8_t *a, const uint8_t *b) {
    for (int i = 0; i < 32; i++)
        if (a[i] != b[i])
            return 0;
    return 1;
}

static char **load(const char *path, int *cnt) {
    FILE *f = fopen(path, "r");
    if (!f) {
        fprintf(stderr, "erro abrindo %s\n", path);
        exit(1);
    }
    int cap = 1024, n = 0;
    char **arr = malloc(cap * sizeof(char *));
    char line[256];
    while (fgets(line, sizeof(line), f)) {
        int L = strlen(line);
        while (L > 0 && (line[L-1] == '\n' || line[L-1] == '\r'))
            line[--L] = 0;
        if (L == 0 || L > 40)
            continue;
        if (n >= cap) {
            cap *= 2;
            arr = realloc(arr, cap * sizeof(char *));
```

```
        }
        arr[n] = strdup(line);
        n++;
    }
    fclose(f);
    *cnt = n;
    return arr;
}

static const char *CONN[5] = {"", "de", "da", "do", "e"};
static volatile int FOUND = 0;

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "uso: %s <mawords> <words> [minlen]\n", argv[0]);
        return 1;
    }
    int minlen = argc > 3 ? atoi(argv[3]) : 10;
    hex2bin(THEX, TARGET);

    int n1, n2;
    char **ma = load(argv[1], &n1);
    char **wd = load(argv[2], &n2);
    printf("mawords=%d words=%d minlen=%d\n", n1, n2, minlen);
    fflush(stdout);

    #pragma omp parallel for schedule(dynamic)
    for (int i = 0; i < n1; i++) {
        if (FOUND)
            continue;
        char buf[128];
        int la = strlen(ma[i]);
        memcpy(buf, ma[i], la);
        for (int j = 0; j < n2; j++) {
            int lw = strlen(wd[j]);
            for (int ci = 0; ci < 5; ci++) {
                int lc = strlen(CONN[ci]);
                int total = la + lc + lw;
                if (total < minlen || total > 55)
                    continue;
                memcpy(buf + la, CONN[ci], lc);
                memcpy(buf + la + lc, wd[j], lw);

                uint8_t d[32];
                sha256_blk((uint8_t *)buf, total, d);
                if (eq(d, TARGET)) {
                    buf[total] = 0;
                    #pragma omp critical
                    {
                        printf("\n>>> ACHOU (sem term) senha=%s\n", buf);
                        fflush(stdout);
                        FOUND = 1;
                    }
                }
                buf[total] = '\n';
                sha256_blk((uint8_t *)buf, total + 1, d);
                if (eq(d, TARGET)) {
                    buf[total] = 0;
                    #pragma omp critical
                    {
                        printf("\n>>> ACHOU (\\n) senha=%s\n", buf);
                        fflush(stdout);
                        FOUND = 1;
                    }
```

```
                }
            }
        }
    }

    if (!FOUND) {
        printf("nada.\n");
        return 2;
    }
    return 0;
}
```

10.2 decrypt_pubkey.py — decifração da chave pública

```
"""Fase 2: dada a SENHA, tenta decifrar key_public.en (cripto simetrica) e
detectar a chave publica RSA. Varre cifras/modos/encodings comuns do OpenSSL.
Uso: python decrypt_pubkey.py "<senha>"
Observacao: key_public.en tem 157 bytes (nao multiplo de 16) -> provavel cifra
de FLUXO (rc4, chacha20, aes-*-ctr/cfb/ofb) OU conteudo base64/pem.
"""
import subprocess, sys, os, base64, binascii

SRC = "/home/deko/Documentos/Facul/5 - AED3/Cripto/key_public.en"
PWD = sys.argv[1] if len(sys.argv) > 1 else None
if not PWD:
    sys.exit("uso: python decrypt_pubkey.py <senha>")

data = open(SRC, "rb").read()
print(f"key_public.en: {len(data)} bytes")

CIPHERS = ["rc4","rc4-40","aes-256-ctr","aes-128-ctr","aes-256-cfb","aes-128-cfb",
           "aes-256-ofb","aes-128-ofb","aes-256-cbc","aes-128-cbc","aes-192-cbc",
           "aes-256-ecb","aes-128-ecb","des-ede3-cbc","des-cbc","bf-cbc",
           "camellia-256-cbc","chacha20","aria-256-cbc","sm4-cbc","seed-cbc"]
MDS = ["md5","sha256","sha1","sha512"]

def looks_like_key(b):
    if b"-----BEGIN" in b and b"KEY-----" in b:
        return "PEM"
    # DER: SEQUENCE
    if b[:1] == b"\x30" and len(b) > 10:
        return "DER?"
    # base64 de PEM/DER
    try:
        dec = base64.b64decode(b.strip(), validate=False)
        if dec[:1] == b"\x30" or b"BEGIN" in dec:
            return "BASE64"
    except Exception:
        pass
    return None

def try_openssl(cipher, md, salt, pbkdf2):
    cmd = ["openssl","enc","-d","-"+cipher,"-pass","pass:"+PWD,"-md",md]
    if not salt: cmd.append("-nosalt")
    if pbkdf2: cmd.append("-pbkdf2")
    try:
        p = subprocess.run(cmd, input=data, capture_output=True, timeout=10)
        if p.returncode == 0 and p.stdout:
            return p.stdout
    except Exception:
        pass
    return None

```

```
# tambem tenta interpretar o arquivo como base64 antes de decifrar
inputs = [("raw", data)]
try:
    b = base64.b64decode(data.strip()); inputs.append(("b64->bin", b))
except Exception: pass

found = []
for label, _ in inputs:
    pass

for cipher in CIPHERS:
    for md in MDS:
        for salt in (False, True):
            for pbk in (False, True):
                out = try_openssl(cipher, md, salt, pbk)
                if out:
                    kind = looks_like_key(out)
                    if kind:
                        print(f"\n>>> POSSIVEL CHAVE cipher={cipher} md={md} salt={salt} pbkdf2={pbk}
                            fmt={kind}")
                        print(out[:200])
                        found.append((cipher,md,salt,pbk,out))
if not found:
    print("\nNenhuma combinacao openssl produziu algo parecido com chave. "
          "Talvez precise de IV explicito, outra ferramenta, ou o conteudo seja raw DER cifrado em
              fluxo.")
else:
    print(f"\n{len(found)} candidata(s). Salvando a 1a em pubkey_candidata.bin")
    open("pubkey_candidata.bin","wb").write(found[0][4])
```

10.3 `factor_rsa.py` — fatoração elementar (pré-GNFS)

```
"""Fatoracao de N (modulo RSA pequeno, ~384 bits) para reconstruir a chave privada.
Estrategias, em ordem: fatores pequenos (trial), Fermat (primos proximos),
Pollard p-1, Pollard rho (Brent). Retorna (p, q).
Tambem utilizavel como: python factor_rsa.py <N>
"""
import sys, math, random

def trial(n, lim=1_000_000):
    f = []
    for p in (2,3,5,7,11,13,17,19,23,29,31,37):
        while n % p == 0:
            f.append(p); n//=p
    i = 41
    while i < lim and i*i <= n:
        while n % i == 0:
            f.append(i); n//=i
        i += 2
    return f, n

def isqrt(n):
    return math.isqrt(n)

def fermat(n, maxiter=10_000_000):
    a = isqrt(n)
    if a*a < n: a += 1
    for _ in range(maxiter):
        b2 = a*a - n
        b = isqrt(b2)
        if b*b == b2:
            return (a-b, a+b)
        a += 1
```

```
    return None

def pollard_pm1(n, bound=100000):
    a = 2
    for j in range(2, bound):
        a = pow(a, j, n)
        if j % 100 == 0:
            d = math.gcd(a-1, n)
            if 1 < d < n:
                return d
    d = math.gcd(a-1, n)
    return d if 1 < d < n else None

def brent(n):
    if n % 2 == 0: return 2
    if n == 1: return 1
    while True:
        y = random.randrange(1, n-1)
        c = random.randrange(1, n-1)
        m = 128; g = q = r = 1
        while g == 1:
            x = y
            for _ in range(r): y = (y*y + c) % n
            k = 0
            while k < r and g == 1:
                ys = y
                for _ in range(min(m, r-k)):
                    y = (y*y + c) % n
                    q = (q * abs(x-y)) % n
                g = math.gcd(q, n); k += m
            r *= 2
        if g == n:
            g = 1
            while g == 1:
                ys = (ys*ys + c) % n
                g = math.gcd(abs(x-ys), n)
        if g != n:
            return g

def is_probable_prime(n):
    if n < 2: return False
    for p in (2,3,5,7,11,13,17,19,23,29,31,37):
        if n % p == 0: return n == p
    d = n-1; r = 0
    while d % 2 == 0: d//=2; r+=1
    for a in (2,3,5,7,11,13,17,19,23,29,31,37):
        x = pow(a, d, n)
        if x in (1, n-1): continue
        for _ in range(r-1):
            x = x*x % n
            if x == n-1: break
        else:
            return False
    return True

def factor(n):
    # 1) fatores pequenos
    fs, rem = trial(n, 100000)
    if rem == 1 and len(fs) == 2:
        return fs[0], fs[1]
    if fs and is_probable_prime(rem):
        return fs[0], rem
    n2 = rem if rem > 1 else n
    # 2) Fermat (primos proximos) - rapido se p~q
```

```
    f = fermat(n2, maxiter=2_000_000)
    if f and f[0] > 1 and f[0]*f[1] == n2:
        return f
    # 3) p-1
    d = pollard_pm1(n2)
    if d and n2 % d == 0:
        return d, n2//d
    # 4) rho (Brent)
    d = brent(n2)
    if d and 1 < d < n2 and n2 % d == 0:
        return d, n2//d
    return None

if __name__ == "__main__":
    if len(sys.argv) > 1:
        N = int(sys.argv[1])
        print("fatorando N de", N.bit_length(), "bits...")
        r = factor(N)
        print("resultado:", r)
        if r: print("verifica:", r[0]*r[1] == N, "primos?", is_probable_prime(r[0]),
            is_probable_prime(r[1]))
```

10.4 `finish.py` — reconstrução da chave privada e decifração final

```
"""Fase final: dados os primos p,q do modulo RSA, reconstroi a chave privada,
decifra key_for_message.en (RSA) -> chave simetrica, e decifra message.en (openssl enc).
Uso: python finish.py <p> <q>
"""
import sys, subprocess, os

BASE = "/home/deko/Documentos/Facul/5 - AED3/Cripto"
KFM = os.path.join(BASE, "key_for_message.en")
MSG = os.path.join(BASE, "message.en")
# N e e extraidos da chave publica decifrada
N = 0
    x25bf10833690feff0336fb7f94e895d0f82d02c461f99671e94546abd3a669a3ef8dc278a7e3e6a8b1c2a582cb55251b

E = 65537

def egcd(a, b):
    if b == 0: return (a, 1, 0)
    g, x, y = egcd(b, a % b); return (g, y, x - (a // b) * y)
def inv(a, m):
    g, x, _ = egcd(a, m); return x % m

def main():
    p, q = int(sys.argv[1]), int(sys.argv[2])
    assert p * q == N, "p*q != N"
    phi = (p - 1) * (q - 1)
    d = inv(E, phi)
    c = int.from_bytes(open(KFM, "rb").read(), "big")
    m = pow(c, d, N)
    k = 48 # tamanho do modulo em bytes
    raw = m.to_bytes(k, "big")
    print("RSA plaintext (hex):", raw.hex())
    # tenta remover padding PKCS#1 v1.5 (00 02 PS 00 M) ou mostra cru
    keys_to_try = []
    if raw[0] == 0x00 and raw[1] == 0x02:
        idx = raw.find(b"\x00", 2)
        if idx > 0:
            payload = raw[idx+1:]
            print("PKCS#1 v1.5 -> payload:", payload.hex(), repr(payload))
            keys_to_try.append(("pkcs1", payload))
```

```
    # tambem tenta: bytes crus sem zeros a esquerda, e repr ascii
    stripped = raw.lstrip(b"\x00")
    keys_to_try.append(("raw_nolead", stripped))
    keys_to_try.append(("raw_full", raw))
    try:
        print("payload como texto:", stripped.decode("utf-8", "replace"))
    except Exception: pass

    data = open(MSG, "rb").read()
    print("message.en:", len(data), "bytes, header:", data[:8])
    salted = data[:8] == b"Salted__"

    CIPHERS = ["rc4","aes-256-ctr","aes-128-ctr","aes-256-cfb","aes-128-cfb",
               "aes-256-ofb","aes-128-ofb","chacha20","aes-256-cbc","aes-128-cbc",
               "des-ede3-cbc","bf-cbc","camellia-256-cbc"]
    MDS = ["sha256","md5","sha1","sha512"]

    for label, key in keys_to_try:
        # a chave pode ser usada como "senha" (pass:) em texto, ou como bytes hex
        passforms = []
        try:
            txt = key.decode("ascii")
            if txt.isprintable(): passforms.append(("ascii", txt))
        except Exception: pass
        passforms.append(("hex", key.hex()))
        for pflabel, passwd in passforms:
            for cph in CIPHERS:
                for md in MDS:
                    for extra in ([], ["-nosalt"]):
                        cmd = ["openssl","enc","-d",f"-{cph}","-pass",f"pass:{passwd}","-md",md,"-in"
                            ,MSG]
                        if "-nosalt" in extra and salted: continue
                        if "-nosalt" not in extra and not salted: cmd += ["-nosalt"]
                        try:
                            out = subprocess.run(cmd, capture_output=True, timeout=10).stdout
                        except Exception: continue
                        if not out: continue
                        printable = sum(1 for b in out if 9 <= b <= 126) / max(len(out),1)
                        if printable > 0.92 and len(out) > 20:
                            print(f"\n>>> POSSIVEL MENSAGEM keyform={label}/{pflabel} cipher={cph} md
                                ={md} {extra}")
                            print(out[:800].decode("utf-8","replace"))
                            open(f"MENSAGEM_{label}_{cph}_{md}.txt","wb").write(out)

    # tambem: pbkdf2
    for label, key in keys_to_try:
        passforms = [("hex", key.hex())]
        try:
            txt = key.decode("ascii")
            if txt.isprintable(): passforms.append(("ascii", txt))
        except Exception: pass
        for pflabel, passwd in passforms:
            for cph in CIPHERS:
                cmd = ["openssl","enc","-d",f"-{cph}","-pbkdf2","-pass",f"pass:{passwd}","-in",MSG]
                try:
                    out = subprocess.run(cmd, capture_output=True, timeout=10).stdout
                except Exception: continue
                if not out: continue
                printable = sum(1 for b in out if 9 <= b <= 126) / max(len(out),1)
                if printable > 0.92 and len(out) > 20:
                    print(f"\n>>> POSSIVEL MENSAGEM (pbkdf2) keyform={label}/{pflabel} cipher={cph}"
                        )
                    print(out[:800].decode("utf-8","replace"))
                    open(f"MENSAGEM_pbkdf2_{label}_{cph}.txt","wb").write(out)
```

```
99
100 if __name__ == "__main__":
101     main()
```

Conclusão

A cadeia completa foi quebrada: `senha=marvinminsky` → chave pública RSA-382 → fatoração por GNFS → `chave=shafigoldwas` → mensagem em claro endereçada ao Grupo 2. O exercício ilustra na prática por que **RSA de baixa entropia** (módulos pequenos e fatoráveis) e **senhas de espaço de busca limitado** (nomes conhecidos) são inseguros, mesmo combinados com AES-256.